app's layout at a minimum. You'll also want to make sure that your Identity Razor Pages use the same base Layout.cshtml as the rest of your application.

We'll start by fixing the layout for your Identity pages. Create a file at the "magic" path `Areas/Identity/Pages/_ViewStart.cshtml`, and add the following contents:

```
@{ Layout = "/Pages/Shared/_Layout.cshtml"; }
```

This sets the default layout for your Identity pages to your application's default layout. Next, add a _LoginPartial.cshtml file in Pages/Shared to define the login widget, as shown in the following listing. This is pretty much identical to the template generated by the default template, but it uses our custom `ApplicationUser` instead of the default `IdentityUser`.

Listing 23.6 Adding a _LoginPartial.cshtml to an existing app

```
@using Microsoft.AspNetCore.Identity
@using RecipeApplication.Data;                          ❶
@inject SignInManager<ApplicationUser> SignInManager    ❷
@inject UserManager<ApplicationUser> UserManager        ❷

<ul class="navbar-nav">
@if (SignInManager.IsSignedIn(User))
{
  <li class="nav-item">
    <a  class="nav-link text-dark" asp-area="Identity"
    asp-page="/Account/Manage/Index" title="Manage">
        Hello @User.Identity.Name!</a>
  </li>
    <li class="nav-item">
      <form class="form-inline" asp-page="/Account/Logout"
      asp-route-returnUrl="@Url.Page("/", new { area = "" })"
      asp-area="Identity" method="post" >
        <button  class="nav-link btn btn-link text-dark"
          type="submit">Logout</button>
        </form>
    </li>
}
else
{
  <li class="nav-item">
    <a class="nav-link text-dark" asp-area="Identity"
      asp-page="/Account/Register">Register</a>
```